

AI Governed Blockchain Architecture for Secure Healthcare IOT Systems

Divya JAYAPRAKASH

Jayasri DHANAPAL

Department of Computer Science and Data Science

Contents

1	Introduction	1
2	Proposed System Architecture	1
2.1	IoT Data Acquisition Layer	1
2.2	Data Preprocessing and Visualisation Layer	2
3	AI Model Training Layer	2
3.1	Isolation Forest	2
3.2	XGBoost Classifier	2
3.3	LSTM Classifier	2
4	Real-Time Inference Pipeline	2
5	Adaptive Confidence-Based Fusion Gate	3
5.1	Fusion Inputs	3
5.2	Fusion Formula	3
6	Database Layer	3
7	Blockchain Security and Encryption Layer	3
7.1	RSA Key Pair	3
7.2	Hybrid Encryption Algorithm	3
7.3	Hash Function	4
7.4	End-to-End Security Workflow	4
7.5	Integrity Verification	4
7.6	Blockchain Storage Design	4
7.7	Smart Contract Functions	4
7.8	Environment Variables	4
7.9	Private Development Network: Hardhat	4
7.10	Public Validation Network: Sepolia Testnet	5
8	Flask Server Layer	5
9	Dashboard Layer	5
10	Experimental Setup	5
11	Implementation and Deployment	5
11.1	Implementation Summary	5
11.2	Running the System End-to-End	5
11.3	Deployment: Hardhat and Sepolia	6
11.4	Configuration	6
12	Results and Observations	6
13	Discussion	6
14	Limitations	6
15	Conclusion	7
16	State-of-the-Art	7
16.1	Temporal Deep Learning for Clinical Classification	7
16.2	Blockchain-Based Healthcare Data Management	7
16.3	Anomaly and Device-Fault Detection	7
16.4	IoT-Based Real-Time Health Monitoring	7
16.5	Gap and Positioning	7
16.6	References	7

Abstract—This paper presents a secure IoT healthcare monitoring framework for real-time vital sign classification and tamper-proof data storage. The system processes readings from IoT sensors and classifies them into `normal_vitals`, `critical_vitals`, and `device_error` using a three-stage AI pipeline: Isolation Forest for unsupervised anomaly and device-fault detection, XGBoost and LSTM for parallel current-reading and sequence-based classification, and an adaptive fusion gate that dynamically weights their outputs via a learned α value. Trained on the PhysioNet/CinC Challenge 2019 dataset, all records are stored in InfluxDB as time-series data. Patient vitals are protected using RSA-OAEP-256 plus AES-256-GCM hybrid encryption; only the encrypted vitals are stored off-chain in InfluxDB while Ethereum anchors only the SHA-256 verification hash and storage reference, ensuring data integrity without exposing sensitive information on-chain.

Keywords—IoT healthcare, medical sensors, vital sign monitoring, machine learning, Isolation Forest, XGBoost, LSTM, adaptive fusion, blockchain, Ethereum, RSA-OAEP, AES-GCM, InfluxDB, secure medical data storage.

1. Introduction

Remote patient monitoring has become an important component of modern healthcare systems. IoT-enabled medical devices can continuously measure vital signs — body temperature, heart rate, oxygen saturation (SpO_2), and respiratory rate — and transmit them to cloud or edge platforms for clinical interpretation. Despite these advantages, IoT healthcare monitoring systems face three practical limitations: (1) sensor readings may be noisy or faulty due to device malfunction or environmental interference; (2) patient criticality cannot always be inferred from a single threshold, as temporal combinations of multiple vitals often indicate deterioration; and (3) medical data require secure storage and controlled access that conventional databases alone cannot guarantee. Blockchain provides immutable record anchoring, but raw patient data should not be stored openly on-chain.

To address these gaps, this work proposes a secure AI-driven IoT healthcare monitoring framework with the following objectives:

- Acquire or simulate vital readings (temperature, heart rate, SpO_2 , respiratory rate) via ESP32 with MAX30102 and DS18B20 sensors, or a Python simulator for demo without hardware.
- Preprocess PhysioNet/CinC Challenge 2019 PSV data and label readings into **normal_vitals**, **critical_vitals**, and **device_error** classes.
- Detect faulty sensor readings using an unsupervised Isolation Forest before applying supervised classification.
- Classify patient vital status using XGBoost (current-reading) and LSTM (12-reading sequence), combined via an adaptive fusion gate (Stage 3).
- Store all records in InfluxDB and encrypt only the four input vitals using RSA-OAEP-256 plus AES-256-GCM hybrid encryption before anchoring verification metadata on Ethereum.
- Provide a React dashboard for live monitoring, JSON testing, database inspection, and blockchain record viewing.

The main novelty of this work is the integration of three-class vital classification, adaptive model fusion, sensor-fault

detection, secure time-series storage, and blockchain integrity protection into a single deployable IoT healthcare architecture. The key contributions are:

- **Three-class classification:** Readings are labelled as **normal_vitals**, **critical_vitals**, or **device_error**, distinguishing patient deterioration from sensor faults.
- **Adaptive Fusion Gate (Stage 3):** A pretrained feed-forward network learns a per-sample α to dynamically weight XGBoost versus LSTM, rather than using fixed voting. It is trained separately, after the base models have already been trained.
- **Fault-aware pipeline:** The system evaluates device-error likelihood before clinical severity, avoiding false critical alerts from faulty sensors.
- **Dual-storage with hybrid encryption:** InfluxDB stores all time-series data including encrypted vitals; Ethereum stores only the SHA-256 hash and InfluxDB storage reference, keeping patient data off-chain at all times.
- **Complete demonstration stack:** Flask API, React dashboard, Python simulator, ESP32 firmware scaffold, InfluxDB, and an Ethereum smart contract developed on a private Hardhat network and validated on the Sepolia testnet, with full deployment, storage, retrieval, and decryption functionality.

2. Proposed System Architecture

The proposed system is organised into nine layers: (1) IoT Data Acquisition, (2) Data Preprocessing and Visualisation, (3) AI Model Training, (4) Real-Time Inference, (5) Adaptive Fusion, (6) Database, (7) Blockchain and Encryption, (8) Flask Server, and (9) Dashboard. The overall data flow is:

- 1) IoT sensor or simulator sends a JSON vital reading to Flask.
- 2) Feature engineering extracts deltas, rolling statistics, and stuck-sensor flags.
- 3) Isolation Forest, XGBoost, and LSTM run in the inference pipeline.
- 4) The Adaptive Fusion Gate combines model outputs into a final label with confidence and trust score.
- 5) All fields are written to InfluxDB.
- 6) Only the four input vitals are encrypted and stored in InfluxDB; hash and metadata are anchored on Ethereum.
- 7) The React dashboard visualises all outputs in real time.

Fig. 1 illustrates the three-stage AI inference pipeline at the core of the system, showing how a new sensor reading passes through Stage 1 (Isolation Forest), Stage 2A/2B (XGBoost and LSTM) and Stage 3 (Adaptive Fusion Gate) to produce the final label and per-class probabilities.

2.1. IoT Data Acquisition Layer

The target hardware setup includes an ESP32 microcontroller, MAX30102 sensor for heart rate and SpO_2 , and DS18B20 temperature sensor; respiratory rate is estimated from sensor signal trends or an external sensor. For demonstration without hardware, a Python-based simulator generates three types of readings: normal vitals, critical vitals, and device-error readings. An example JSON payload submitted to the Flask API is:

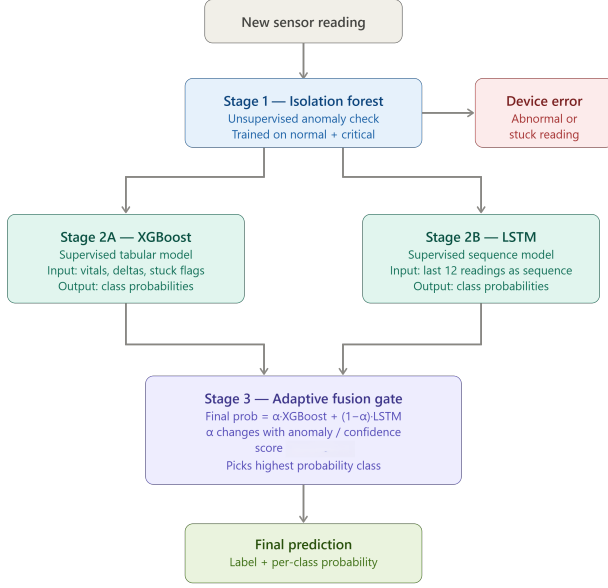


Fig. 1. Three-stage AI inference pipeline: Stage 1 (Isolation Forest) performs unsupervised anomaly detection and flags device errors; Stage 2A (XGBoost) and Stage 2B (LSTM) classify the reading in parallel; Stage 3 (Adaptive Fusion Gate) dynamically weights their outputs via α to produce the final label and per-class probabilities.

```
{
  "temperature": 39.4,
  "heart_rate": 135,
  "spo2": 88,
  "respiratory_rate": 31,
  "device_id": "ESP32-SIM-01",
  "patient_id": "patient-demo",
  "timestamp": "2026-07-01T10:00:00Z"
}
```

2.2. Data Preprocessing and Visualisation Layer

The preprocessing stage reads the PhysioNet/CinC Challenge 2019 PSV records, retains only the four target vitals, labels and balances the three classes, and produces a cleaned dataset ready for model training. The feature mapping from dataset columns to project features is shown in Table I.

TABLE I
FEATURE MAPPING FROM DATASET TO PROJECT

Original Column	Project Feature
Temp	temperature
HR	heart_rate
O2Sat	spo2
Resp	respiratory_rate

The full preprocessing steps are: (1) load the raw PSV records; (2) select the four target vitals; (3) convert values to numeric format; (4) interpolate missing values per patient; (5) fill remaining missing values using median values; (6) generate temporal features including vital deltas, rolling 12-reading mean, minimum, and maximum; (7) generate clinical labels (**normal_vitals**, **critical_vitals**, **device_error**); (8) simulate device-error rows if real examples are unavailable;

(9) balance all three classes; and (10) save the final cleaned dataset. Visualisation outputs include class distribution and per-vital trend plots.

3. AI Model Training Layer

All three base models are trained together in a single training stage. The Stage 3 adaptive fusion gate is trained separately afterward, once the base models are available.

3.1. Isolation Forest

The Isolation Forest is an unsupervised anomaly detection algorithm trained exclusively on **normal_vitals** and **critical_vitals**; device-error samples are intentionally excluded so that impossible physiological readings appear statistically anomalous at inference time. Input features include original vitals, delta features, and rolling-window features. The contamination parameter is set to 0.05. The output is a binary **device_error_flag** and a continuous anomaly score, both passed forward in the pipeline without halting processing.

3.2. XGBoost Classifier

XGBoost is trained on all three classes using 12 input features (raw vitals, delta features, and stuck-sensor flags). Class imbalance is handled via inverse-frequency class weights. Training runs for up to 500 boosting rounds with early stopping at 30 rounds of no improvement. A representative output probability vector is:

```
{
  "normal_vitals": 0.12,
  "critical_vitals": 0.84,
  "device_error": 0.04
}
```

3.3. LSTM Classifier

The LSTM model operates on sliding windows of 12 consecutive readings (**SEQUENCE_LEN = 12**) per patient. Input features per time-step are the four raw vitals and their delta values, giving an 8-dimensional feature vector. Configuration: hidden size 64, 2 layers, batch size 256, 30 epochs, learning rate 10^{-3} . If fewer than 12 readings are available at runtime, XGBoost probabilities serve as the LSTM fallback and the sequence availability flag is set to false. A representative LSTM output is:

```
{
  "normal_vitals": 0.20,
  "critical_vitals": 0.75,
  "device_error": 0.05
}
```

4. Real-Time Inference Pipeline

When a new IoT reading arrives at **POST /api/vitals**, the following steps execute in sequence:

- 1) The payload is normalised.
- 2) Feature engineering is applied: deltas, rolling statistics, and stuck-sensor flags.
- 3) Isolation Forest calculates the anomaly score and **device_error_flag**.

- 4) XGBoost predicts class probabilities from the current reading.
- 5) LSTM predicts class probabilities from the 12-reading sequence if available.
- 6) Adaptive fusion gate computes α and generates fused class probabilities.
- 7) Final label, confidence score, and trust score are produced.
- 8) All fields are written to InfluxDB.
- 9) Only heart rate, temperature, respiratory rate, and SpO₂ are encrypted with the doctor public key.
- 10) Encrypted vitals are stored in InfluxDB; hash and meta-data are anchored on Ethereum.

The system always writes to InfluxDB first; Ethereum metadata storage is attempted only after a successful InfluxDB write. If the InfluxDB write fails, blockchain storage is stopped.

5. Adaptive Confidence-Based Fusion Gate

The Adaptive Confidence-Based Fusion Gate is Stage 3 of the pipeline and the key novel contribution of this work. It is a pretrained feed-forward neural network that learns α from per-sample evidence rather than using fixed ensemble weights.

5.1. Fusion Inputs

The gate is trained on: original vitals (temperature, heart rate, SpO₂, respiratory rate); XGBoost probability vector; LSTM probability vector; Isolation Forest anomaly score; XGBoost confidence and entropy; LSTM confidence and entropy; and a sequence availability flag.

5.2. Fusion Formula

Let $\alpha \in [0, 1]$ denote the dynamic weight assigned to XGBoost. The final class probability vector is:

$$P_{\text{final}} = \alpha \cdot P_{\text{XGBoost}} + (1 - \alpha) \cdot P_{\text{LSTM}} \quad (1)$$

Higher α trusts XGBoost more (stable current readings); lower α trusts LSTM more (temporal sequence patterns). The final prediction is the class with the highest fused probability.

For example, with XGBoost predictions {Normal: 0.05, Critical: 0.92, Device Error: 0.03}, LSTM predictions {Normal: 0.12, Critical: 0.85, Device Error: 0.03}, and $\alpha = 0.6$:

$$P_{\text{Normal}} = 0.6(0.05) + 0.4(0.12) = 0.078 \quad (2)$$

$$P_{\text{Critical}} = 0.6(0.92) + 0.4(0.85) = 0.892 \quad (3)$$

$$P_{\text{DeviceError}} = 0.6(0.03) + 0.4(0.03) = 0.030 \quad (4)$$

Final Prediction: *Critical* (highest fused probability 0.892).

6. Database Layer

InfluxDB is used as the time-series database. Each reading is stored as a **medical_vitals** measurement containing fields: **temperature**, **heart_rate**, **spo2**, **respiratory_rate**, **anomaly_score**, **alpha**, **confidence**, **trust_score**, per-model probability outputs (**xgb_***, **lstm_***, **fusion_***), and **encrypted_vitals** (the base64-encoded AES-GCM encrypted vitals envelope). Tags include **device_id**, **patient_id**, and

final_label. The **storage_id** blockchain reference uses the format:

```
influxdb://<bucket>/medical_vitals/<data_hash>
```

7. Blockchain Security and Encryption Layer

This layer protects patient vitals before blockchain storage. Only the four input vitals are encrypted and stored off-chain in InfluxDB; the Ethereum blockchain stores only verification and lookup metadata. The blockchain layer itself spans two Ethereum networks: a private local network created with Hardhat for development and testing, and the public Sepolia testnet for validating the deployed contract in a real, publicly auditable environment before any future production deployment.

7.1. RSA Key Pair

The asymmetric key pair is stored locally as a public key file and a private key file, in PEM format. The backend auto-generates the pair if it does not already exist, using RSA 2048-bit with public exponent 65537. Table II summarises the key usage.

TABLE II
RSA KEY PAIR USAGE

Key	Used for	Who holds it
Public key	Encrypts vitals	Shared with the system
Private key	Decrypts vitals	Kept secret by doctor/hospital

7.2. Hybrid Encryption Algorithm

The project uses hybrid asymmetric encryption: **RSA-OAEP-256 + AES-256-GCM**. RSA alone is not suitable for larger JSON payloads, so AES-256-GCM encrypts the vitals JSON and RSA-OAEP encrypts the AES key with the doctor public key. Only the doctor private key can recover the AES key, which then decrypts the vitals. The encrypted output is a JSON envelope:

```
{
  "alg": "RSA-OAEP-256+A256GCM",
  "encrypted_key": "...",
  "nonce": "...",
  "ciphertext": "..."
}
```

This envelope is base64-encoded and stored in InfluxDB as the **encrypted_vitals** field. The vitals covered by encryption are:

```
{
  "heart_rate": 85,
  "temperature": 37.1,
  "respiratory_rate": 18,
  "spo2": 98
}
```

7.3. Hash Function

The project uses SHA-256 to generate the blockchain verification hash:

$$\text{dataHash} = \text{SHA-256}(\text{encrypted_vitals}) \quad (5)$$

The hash is computed from the encrypted vitals, not from plaintext. Ethereum stores this hash to verify that the encrypted InfluxDB data has not been modified.

7.4. End-to-End Security Workflow

The seven-step security workflow is as follows.

Step 1 — Key pair creation. The doctor or hospital creates or auto-generates an RSA 2048-bit key pair. The public key is shared with the system; the private key is kept secret.

Step 2 — Data collection. Patient data arrives from the IoT device or simulator. An example payload:

```
{
  "patient_id": "P101",
  "heart_rate": 85,
  "temperature": 37.1,
  "prediction": "normal_vitals"
}
```

Step 3 — Hybrid encryption. The backend encrypts only the four input vitals using AES-256-GCM, then encrypts the AES key using RSA-OAEP with the doctor's public key. The resulting JSON envelope is base64-encoded.

Step 4 — Hash generation. $\text{dataHash} = \text{SHA-256}(\text{encrypted_vitals})$

Step 5 — InfluxDB storage. The system writes to InfluxDB first: all dashboard fields, the **encrypted_vitals** value, and the **storage_id** reference. Ethereum storage is attempted only after a successful InfluxDB write.

Step 6 — Ethereum metadata storage. The smart contract stores the metadata fields listed in Table III.

TABLE III
ETHEREUM SMART CONTRACT STORED FIELDS

Field	Meaning
patientId	Patient identifier, e.g. P101
dataHash	SHA-256 hash of the encrypted vitals
storageId	InfluxDB encrypted vitals reference
doctorWallet	Doctor/hospital Ethereum wallet address
timestamp	Unix timestamp of the reading
finalLabel	Final AI classification label
trustScoreBps	Trust score in basis points

Step 7 — Doctor decryption. The doctor retrieves encrypted vitals via **storage_id** and decrypts:

$$\text{AES_Key} = \text{RSA-OAEP_decrypt}(\text{Enc_AES_Key}, \text{Private_Key}) \quad (6)$$

$$\text{Vitals} = \text{AES-GCM_decrypt}(\text{Enc_Data}, \text{AES_Key}) \quad (7)$$

Decryption can be triggered via the dashboard (*Blockchain Records* tab → Decrypt) or via the API endpoint `/api/blockchain-records?decrypt=1`, both of which use the doctor's private key to recover the original vitals.

7.5. Integrity Verification

To verify a stored record: (1) copy **encrypted_vitals** from InfluxDB; (2) compute SHA-256 of the encrypted string; (3) compare with **dataHash** from Ethereum; (4) if values match, decrypt using the private key. A match proves the off-chain encrypted record has not been modified since the blockchain transaction was written.

7.6. Blockchain Storage Design

The Ethereum smart contract stores metadata only, defending against three attack vectors: (1) Data tampering — SHA-256 hash detects any post-hoc modification; (2) Replay attacks — Unix timestamps and nonces prevent replaying valid transactions; (3) Sybil attacks — permissioned membership limits participation to authenticated healthcare entities.

7.7. Smart Contract Functions

The deployed smart contract exposes three core functions: **storeRecord()**, which stores a patient's metadata on-chain; **recordCount()**, which returns the total number of records stored; and **getRecord()**, which retrieves the metadata for a specific record. Only the authorised owner (the backend) may invoke **storeRecord()**, restricting who can write new entries, while all stored records remain publicly readable, auditable, and immutable once written.

7.8. Environment Variables

All blockchain parameters are configured through environment-specific configuration variables. Table IV lists each variable with an example value and its function.

TABLE IV
BLOCKCHAIN ENVIRONMENT VARIABLES

Variable	Function
RPC_URL	Ethereum RPC endpoint (local Hardhat network or Sepolia testnet)
WEB3_PROVIDER_URL	Web3 provider URL for Flask backend
ETH_PRIVATE_KEY	Wallet private key that pays gas
CONTRACT_ADDRESS	Deployed smart contract address (Hardhat or Sepolia)
DOCTOR_WALLET	Doctor/hospital wallet address stored on-chain
PUBLIC_KEY_PATH	Location of the public key
PRIVATE_KEY_PATH	Location of the private key

If **DOCTOR_WALLET** is empty, the backend derives the wallet from **ETH_PRIVATE_KEY**. If no Ethereum key is configured, records are still encrypted and stored in InfluxDB but **stored_on_chain** is set to **false**.

7.9. Private Development Network: Hardhat

During development, the smart contract was deployed and exercised on a private Ethereum network created with the Hardhat development framework. Hardhat provides a local Ethereum network, reachable only on the machine running it, that supports full smart contract deployment and transaction execution without touching the public Ethereum network. This private network is free to use, fast, isolated from external traffic, and well suited to iterative development, debugging, and research ahead of any public deployment.

7.10. Public Validation Network: Sepolia Testnet

Once the contract’s behaviour was validated on the private network, it was deployed to the Sepolia public testnet, which mimics Ethereum Mainnet behaviour while using free test ETH rather than real cryptocurrency. Sepolia provides a shared, publicly auditable environment in which the deployed contract, and every transaction it records, can be independently verified through Sepolia Etherscan. Gas fees for contract deployment and testing on Sepolia were covered using a Google Cloud Web3 faucet, which funded the sender wallet with test ETH. For every `/api/vitals` request processed against Sepolia, the backend encrypted the four input vitals, stored the encrypted vitals in InfluxDB, and anchored **patientId**, **dataHash**, **storageId**, **doctorWallet**, timestamp, label, and trust score on Sepolia, saving the resulting transaction hash back into InfluxDB. Validating on Sepolia in this way confirms data integrity and transparency on a real, publicly verifiable Ethereum network, ahead of any future production deployment to the Ethereum Mainnet.

8. Flask Server Layer

The Flask backend provides REST APIs as summarised in Table V.

TABLE V
FLASK REST API ENDPOINTS

Endpoint	Purpose
GET /health	Check backend and InfluxDB status
POST /api/vitals	Submit vitals and receive prediction
GET /api/latest	Get latest prediction
GET /api/db-records	Read records from InfluxDB
GET /api/db-status	Check InfluxDB connection status
POST /api/blockchain-store	Encrypt and store latest record
GET /api/blockchain-records	View blockchain records
GET /api/models/status	Check model artifact status
POST /api/simulator/start	Start IoT simulator
POST /api/simulator/stop	Stop IoT simulator

9. Dashboard Layer

The React dashboard (built with Vite) provides four tabs:

- 1) **Live Monitoring** — displays latest vitals, label, trust score, α , anomaly score, and XGBoost, LSTM, and fusion probabilities in real time.
- 2) **Test Prediction** — allows the user to paste sample JSON, view full model output, then click *Encrypt and Store Blockchain* to encrypt and anchor the prediction.
- 3) **Database Records** — shows InfluxDB records with vitals, label, confidence, anomaly score, α , trust score, blockchain transaction hash, and an *Encrypted Vitals* column (shortened in display; full value available on hover or via API).
- 4) **Blockchain Records** — shows Ethereum metadata: **ethereumRecordId**, **patientId**, **dataHash**, **storageId**, **doctorWalletAddress**, **timestamp**, **finalLabel**, and **trustScore**. Supports optional local decryption via `/api/blockchain-records?decrypt=1`.

10. Experimental Setup

The implementation uses the technology stack shown in Table VI.

TABLE VI
IMPLEMENTATION TECHNOLOGY STACK

Component	Technology
Backend	Flask (Python)
Frontend	React + Vite
Machine learning	scikit-learn, XGBoost, PyTorch
Time-series DB	InfluxDB
Blockchain	Ethereum (Hardhat private network, Sepolia testnet), Solidity
Encryption	RSA-OAEP-256, AES-256-GCM
Simulator	Python
IoT hardware	ESP32 + MAX30102 + DS18B20

Model artefacts produced by training include the trained Isolation Forest, XGBoost, LSTM, and adaptive fusion models, together with their associated scalers and metadata.

11. Implementation and Deployment

This section describes how the components introduced in the previous sections are implemented in software and how the complete system is deployed and run end to end. The blockchain layer was developed and tested on a private Ethereum network created with Hardhat, then validated on the public Sepolia testnet ahead of any future production deployment.

11.1. Implementation Summary

The system is implemented as a modular Python/Flask backend paired with a React frontend, as summarised earlier in Table VI. Model training produces the trained Isolation Forest, XGBoost, LSTM, and adaptive fusion models, together with their associated scalers and metadata. At runtime, the Flask server (Table V) loads these trained models once and reuses them for every incoming reading, keeping inference stateless apart from the rolling per-device window maintained for the LSTM.

11.2. Running the System End-to-End

A full run of the system involves four components operating together:

- 1) **InfluxDB** — the time-series database that persists every reading, model output, and encrypted-vitals envelope.
- 2) **Ethereum smart contract** — deployed either on the private Hardhat network during development or on the Sepolia testnet for public validation, anchoring the SHA-256 verification hash and storage reference for each record.
- 3) **Flask backend** — exposes the REST API, runs the three-stage inference pipeline, performs hybrid encryption, and coordinates InfluxDB and Ethereum writes.
- 4) **Data source** — either the Python simulator generating normal, critical, and device-error readings, or the ESP32 firmware streaming live sensor data.

The React dashboard connects to the Flask API and provides the four tabs described in the Dashboard Layer section for live

monitoring, manual test submissions, and inspection of both InfluxDB and Ethereum records.

11.3. Deployment: Hardhat and Sepolia

Development and initial testing were carried out on a private Ethereum network created with Hardhat, which behaves like Ethereum but runs locally, free of charge, and isolated from the public network, making it well suited to rapid iteration. Once the smart contract's behaviour was validated locally, it was deployed to the Sepolia public testnet so that stored verification data could be independently inspected on a real, publicly auditable ledger, without requiring real cryptocurrency; gas fees for Sepolia deployment and testing were covered using a Google Cloud Web3 faucet. In both cases, every `/api/vitals` request encrypted the four input vitals, stored the encrypted vitals in InfluxDB, and anchored `patientId`, `dataHash`, `storageId`, `doctorWallet`, timestamp, label, and trust score on the target network, saving the resulting transaction hash back into InfluxDB. Records anchored on Sepolia can additionally be inspected on the public Sepolia block explorer using either the contract address or an individual transaction hash.

11.4. Configuration

All environment-specific parameters — the RPC endpoint (pointing to either the local Hardhat network or Sepolia), wallet keys, contract address, doctor wallet, and RSA key locations — are configured through the environment variables summarised in Table IV, allowing the same codebase to move between local development, public testnet validation, and (with appropriate hardening) production deployment without code changes.

12. Results and Observations

The proposed secure AI-driven IoT healthcare monitoring framework was successfully implemented and integrated with machine learning, secure database storage, and blockchain-based integrity verification. The following observations were obtained during system execution:

- Incoming patient vital signs were successfully classified into `normal_vitals`, `critical_vitals`, and `device_error` categories.
- The Isolation Forest model effectively identified anomalous sensor readings before supervised classification, reducing the possibility of false clinical alerts caused by faulty devices.
- XGBoost generated immediate predictions from the current vital signs, while the LSTM model performed temporal analysis after sufficient sequential readings were available. During the initial prediction stage, XGBoost served as the primary classifier until sequence data became available for LSTM inference.
- The Adaptive Fusion Gate dynamically combined the prediction probabilities of XGBoost and LSTM by learning an appropriate weighting factor (α) based on model confidence and sequence availability.
- All patient records, prediction results, confidence scores, anomaly scores, and encrypted vital signs were successfully stored in InfluxDB as time-series data.

- Only the blockchain verification metadata, including `dataHash` and `storageId`, was stored on the Ethereum blockchain, ensuring that sensitive patient information remained securely stored off-chain.
- Integrity verification was successfully performed by recomputing the SHA-256 hash of the stored `encrypted_vitals` in InfluxDB and comparing it with the corresponding `dataHash` stored on the blockchain. Matching hash values confirmed that the encrypted medical records had not been modified after storage.
- The complete framework demonstrated the successful integration of IoT data acquisition, machine learning-based health monitoring, hybrid encryption, secure time-series storage, and blockchain-based integrity verification within a unified healthcare monitoring system.

13. Discussion

This framework demonstrates how AI and blockchain can be combined for secure IoT medical monitoring. Isolation Forest provides fault-awareness; XGBoost provides strong classification for current readings; and LSTM provides temporal pattern recognition. The adaptive fusion gate improves upon fixed voting by dynamically assigning per-sample trust to each model.

The database and blockchain design cleanly separates operational monitoring from integrity protection. InfluxDB stores all operational data including the base64-encoded encrypted vitals envelope, while Ethereum provides tamper-evident anchoring through the SHA-256 hash and InfluxDB storage reference. The RSA-OAEP-256 plus AES-256-GCM hybrid scheme ensures that sensitive vitals are never exposed on-chain and that only the authorised doctor holding the private key can recover the original readings.

Developing first on a private Hardhat network and then validating on the Sepolia public testnet demonstrates that the same contract and backend flow can move from free, fast, local iteration to anchoring verification data on a real, publicly auditable Ethereum network, without requiring real cryptocurrency at either stage. The auto-generation of RSA key pairs ensures usability for demos while leaving production deployments free to substitute a hardware security module.

14. Limitations

This project is a prototype with the following limitations:

- Generated labels are rule-based and derived from selected vitals rather than direct clinical diagnosis.
- Device-error examples are simulated rather than collected from real faulty hardware.
- Clinical validation with real hospital deployment has not been performed.
- The blockchain layer stores only metadata and references, not raw medical data.
- The ESP32 firmware requires hardware calibration before real clinical use.
- Model performance depends on preprocessing assumptions and dataset quality.

- The private key is stored locally for demo decryption only; production deployments require a hardware security module or equivalent key management solution.

The system should not be used for real medical decision-making without clinical validation, regulatory review, and deployment hardening.

15. Conclusion

This work presented a layered secure IoT healthcare monitoring framework integrating medical sensor data acquisition, AI-based three-class classification, adaptive fusion, time-series storage, RSA-OAEP-256 plus AES-256-GCM hybrid encryption, and Ethereum blockchain anchoring. Only the four input vitals are encrypted with the doctor public key and stored off-chain in InfluxDB; Ethereum stores only the SHA-256 verification hash and InfluxDB storage reference, keeping sensitive patient data off-chain at all times. The adaptive fusion gate (Stage 3) provides a flexible, learnable method for combining current-reading and temporal-sequence model outputs. The complete implementation demonstrates the feasibility of combining IoT, machine learning, InfluxDB, and Ethereum blockchain for secure medical monitoring applications.

Future work can include real sensor deployment, larger clinical evaluation, improved device-fault modelling, explainable AI integration, blockchain gas optimisation, hardware security module integration for RSA key management, and integration with healthcare interoperability standards such as HL7/FHIR.

16. State-of-the-Art

Research in IoT healthcare monitoring spans four broad areas: temporal deep learning for clinical classification, blockchain-based medical data management, anomaly detection, and IoT sensor frameworks. This section reviews representative works in each area, identifies the gap that motivates the proposed system, and lists the corresponding references.

16.1. Temporal Deep Learning for Clinical Classification

LSTM-based architectures have demonstrated strong performance in detecting sepsis and other acute deterioration events from multi-parameter ICU time-series data [1]. Ensemble LSTM designs that combine multiple recurrent sub-networks improve robustness over single-model approaches. More recent deep learning frameworks for ICU patient monitoring [4] further extend temporal modelling to multi-task clinical prediction. However, these works operate on offline historical EHR data, do not integrate unsupervised device-fault detection, and provide no mechanism for tamper-proof storage of real-time streaming readings.

16.2. Blockchain-Based Healthcare Data Management

Several works have applied blockchain to protect medical record integrity. Secure healthcare data management schemes using Ethereum [2] and privacy-preserving record sharing protocols [3] demonstrate that distributed ledgers can enforce access control and provide auditability. However, these systems store data directly on-chain or rely on simple hash anchoring without combining AI-driven classification, hybrid

asymmetric encryption, or time-series off-chain storage. Predictive analytics work in critical care [5] further explores machine learning for clinical risk scoring but does not address blockchain integration or real-time IoT data streams.

16.3. Anomaly and Device-Fault Detection

Unsupervised anomaly detection using combinations of autoencoders and Isolation Forests [7] has been shown to identify unusual patterns in sensor data without labelled fault examples. These approaches are well suited to detecting impossible or stuck sensor readings that indicate hardware malfunction rather than genuine clinical deterioration. The proposed system incorporates Isolation Forest as Stage 1 specifically to separate device errors from clinical anomalies before applying supervised classification, a design choice not present in prior clinical monitoring works.

16.4. IoT-Based Real-Time Health Monitoring

IoT frameworks combining cloud storage with real-time vital sign streaming [6] improve data availability and remote access but rely on centralised cloud infrastructure that introduces single points of failure and privacy risks. The PhysioNet/CinC Challenge 2019 dataset [8], derived from real ICU patients across multiple hospital systems, provides a widely used benchmark for evaluating clinical classification models on multi-parameter vital sign time-series.

16.5. Gap and Positioning

None of the surveyed works provides a unified, deployable pipeline that combines unsupervised device-fault detection, supervised multi-model classification, adaptive fusion, hybrid RSA-OAEP-256 plus AES-256-GCM encryption, InfluxDB time-series off-chain storage, and Ethereum metadata anchoring in a single framework. The proposed system addresses this gap.

16.6. References

- [1] Author(s), "An Ensemble LSTM Architecture for Clinical Sepsis Detection," *ResearchGate*, 2020. [Online]. Available: <https://www.researchgate.net/publication/338626534>
- [2] Author(s), "Blockchain-based Secure Healthcare Data Management," *Computers in Biology and Medicine*, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0010482520304418>
- [3] Author(s), "Privacy-Preserving Medical Record Sharing Using Blockchain," *Computers in Biology and Medicine*, 2020. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0010482520302912>
- [4] Author(s), "Deep Learning for ICU Patient Monitoring," *arXiv*, 2025. Available: <https://arxiv.org/html/2512.24253v1>
- [5] Author(s), "Predictive Analytics in Critical Care," *Springer Knowledge and Information Systems*, 2025. Available: <https://link.springer.com/article/10.1007/s10115-025-02580-6>
- [6] Author(s), "IoT-based Real-Time Health Monitoring Framework," *Engineering, Technology & Applied Science Research (ETASR)*, vol. X, 2025. Available: <https://etasr.com/index.php/ETASR/article/view/15288>
- [7] Author(s), "Anomaly Detection using Combination of Autoencoder and Isolation Forest," *ResearchGate*, 2023. Available: <https://www.researchgate.net/publication/369254015>
- [8] A. Reyna et al., "Early Prediction of Sepsis from Clinical Data: The PhysioNet/Computing in Cardiology Challenge 2019," in *Proc. CinC*, 2019.